

OntoDAG: A Semantic Layer for Distributed Storage

Péter Földiák

peter.foldiak@gmail.com

<https://github.com/petfold/ontodag>

Abstract

Content-addressed storage networks such as Ethereum Swarm solve integrity and availability, but they say nothing about *meaning*: a 32-byte reference does not tell you what a piece of content is, and the manifests layered on top of it reproduce the fifty-year-old directory tree, which forces every object into exactly one place. We present **OntoDAG**, a multi-parent category DAG that acts as a semantic layer for decentralized storage. Items are stored under any number of categories, and a single query primitive returns everything below *all* of a set of categories. Because the transitive reduction of a DAG is unique, a correctly maintained OntoDAG has exactly one canonical form; combined with a deterministic encoding, the same knowledge always hashes to the same root reference. This makes versions immutable snapshots, makes structural diffs cheap, and lets concurrent, uncoordinated writers merge convergently in the manner of a CRDT, so personal categories can extend shared ontologies without corrupting them. Finally, we show how the category structure itself can be *learned* from data by the minimum description length (MDL) principle: a concept is admitted only if it compresses the data enough to pay for its own description. Learned categories are regenerable, so in Swarm’s postage-stamp economy they behave like a cache — kept warm only while query traffic justifies their storage rent. A Python implementation of the data structure, its persistence layer, and its Swarm adapter is open source and has been validated against a live Bee node on Gnosis mainnet.

1 Introduction

Decentralized storage has made remarkable progress on the questions of *where* and *whether*: content addressing guarantees that a reference identifies exactly one immutable byte string [11, 2], and networks such as Ethereum Swarm [15] add incentivized availability, mutable pointers (feeds), and payment for storage (postage stamps). What none of this provides is an answer to *what*: what is this content, what is it about, how does it relate to everything else I have stored? A hash is the perfect name for a machine and the worst possible name for a mind.

The conventional answer is a manifest: a mapping from path names to references — in other words, a directory tree. But the directory tree is an abstraction

inherited from physical filing, and it systematically under-describes content: every object must live in exactly one place, even though almost everything we store is legitimately *several things at once*. The relational database, the other dominant organizational tool [4], fails for the complementary reason: its fixed columns suit large collections of homogeneous records, not an open-ended universe of heterogeneous things. What is missing between the two is a *semantic layer*: a small, principled structure that records what things are, independently of where their bytes live.

This paper describes OntoDAG [6], a semantic layer designed specifically for content-addressed storage. It makes four contributions:

1. **A minimal data structure** (Section 3): a multi-parent category DAG with two primitives — **put** an item under a set of categories, **get** the intersection of the categories’ descendants — maintained in transitively reduced form, which is provably unique [1] and therefore canonical.
2. **A persistence mapping onto Swarm** (Section 4): one record per node in a canonically encoded trie, so that the same graph always yields the same root hash; commits are atomic root swaps, versions are free, and structural sharing makes updates proportional to the change, not to the graph.
3. **Coordination-free multi-writer merge** (Section 5): defining the canonical state as the transitive reduction of the union of asserted edges makes concurrent updates commutative and idempotent — the defining properties of a conflict-free replicated data type [14].
4. **A learning loop** (Sections 6–7): categories can be proposed automatically by a minimum description length learner [5], and because learned categories are regenerable, Swarm’s storage economics gives them a natural lifecycle — they are a cache, priced by postage.

The intended audience of this paper is developers of decentralized systems, not machine-learning specialists; we introduce the classification and MDL background from first principles.

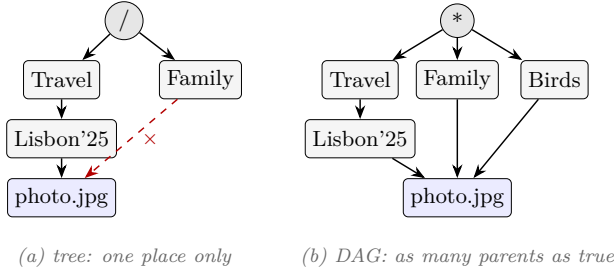


Figure 1: A photograph of a seagull taken on a family trip is *simultaneously* travel, family, and birds. A directory tree forces a choice; a multi-parent DAG records all three memberships, and the item becomes findable from every direction.

2 Why trees and tables are not enough

Directory trees. A tree admits exactly one path to every object, so filing means choosing *the* place a thing belongs — a choice that is almost always a distortion (Figure 1). Is the photo from the family trip filed under **Travel/Lisbon** or under **Family**? Whichever is chosen, the other access path is destroyed. Symbolic links and shortcuts are ad-hoc apologies for this defect, unmanaged and invisible to queries. The deeper problem is that a tree conflates two independent questions — where content is stored and what it means — which is precisely the coupling content addressing was supposed to remove.

Flat tags. Tagging fixes multiple membership but discards structure: tags have no relationship *to each other*. A search for **bird** does not return items tagged **penguin**, because nothing records that a penguin is a bird. Every query must enumerate synonyms and specializations by hand, and the tag vocabulary drifts into an unmanageable soup.

Relational tables. The relational model [4] is superb for large sets of homogeneous records, but its fixed columns are the wrong shape for describing an open universe of heterogeneous things. A car advertisement, a holiday photo, a research paper, and a membership certificate do not share a schema; modelling them relationally means either a table per kind (with the taxonomy frozen into DDL) or a sparse megatable of NULLs. Moreover, subsumption — the single most useful semantic relation, “every X is a Y” — is not a join; SQL can store it but cannot naturally query with it.

Hierarchical and faceted classification. Library science met this problem a century before file systems. *Hierarchical* schemes such as Dewey give a single deep taxonomy — navigable and browsable, but again one shelf position per book. *Faceted* classification, introduced by Ranganathan’s Colon Classification [12], instead describes an object by independent aspects

(his classics: personality, matter, energy, space, time; a modern e-commerce equivalent: product type $\times$ region $\times$ price band $\times$ condition). Facets compose — “electric cars in Germany” — without anyone having pre-built that shelf. But facets alone are not enough either, because each facet is itself hierarchical: *Germany* sits inside *Europe*, *electric* inside *powertrain*. What is actually needed is the combination: **multiple, individually hierarchical facets, whose combinations are formed on demand at query time**. Formally, categories ordered by subsumption with multiple parents form a lattice rather than a tree; Formal Concept Analysis (FCA) [17, 7] studies exactly these structures, and we return to it in Section 6 as the mathematical basis of *learning* categories. The data structure that represents such an order with no redundancy is a directed acyclic graph — which is what OntoDAG is.

3 OntoDAG: the data structure

OntoDAG [6] is a deliberately minimal, subsumption-only ontology: a multi-parent category DAG with one insertion primitive and one query primitive. In the spirit of Bush’s associative memory [3], everything is reachable by what it *is* rather than by where it was put.

Structure. Nodes are *items*. There is intentionally no separate notion of “class” versus “instance”: any instance can later be subdivided (a book $\rightarrow$ an edition $\rightarrow$ my copy $\rightarrow$ my copy when new), so no principled boundary exists, and OntoDAG does not pretend to draw one. An item that tags stored content simply carries a payload reference. Edges mean subsumption: an edge $A \rightarrow B$ asserts that B is a kind of (or an instance of, or a part of the extension of) A . A distinguished root $*$ is the implicit ancestor of everything.

The two primitives.

- **put**($x, \{S_1, \dots, S_k\}$) — insert item x under supercategories $S_1, \dots, S_k$.
- **get**($\{C_1, \dots, C_k\}$) = $\bigcap_{i=1}^k \text{desc}(C_i)$ — return every item below *all* of the query categories, where $\text{desc}(C)$ is the set of descendants of C .

That is the whole interface. A query is a conjunction of facets, and the answer is the intersection of their descendant cones (Figure 2).

Canonical form: the transitive reduction. The stored graph is kept *transitively reduced*: if a path $A \rightarrow B \rightarrow C$ exists, the direct edge $A \rightarrow C$ must not. Redundant supercategory links are pruned automatically — if a user writes **put**(penguin, {Bird, Animal}) while Bird is already below Animal, the Animal $\rightarrow$ penguin edge is dropped as implied. The decisive mathematical fact is that **the transitive reduction of a DAG**

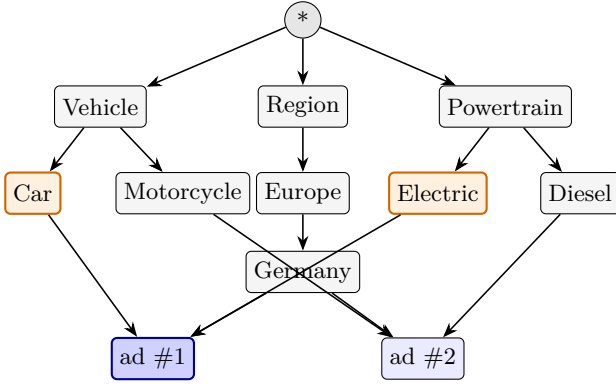


Figure 2: A miniature car-market OntoDAG (taken from the reference implementation’s demo). Listings were inserted with `put(ad#1, {Car, Germany, Electric})` and `put(ad#2, {Motorcycle, Germany, Diesel})`. The query `get({Car, Electric})` (highlighted categories) intersects the two descendant cones and returns `ad #1` (highlighted item) — no “electric cars” category was ever created.

is unique [1]. Uniqueness means a correctly maintained OntoDAG has exactly *one* form regardless of the order or history of insertions: the graph is a pure function of the knowledge it encodes. The reference implementation enforces this with a machine-checked invariant suite — acyclicity, exact transitive reduction, insertion-order independence, consistency of cached descendant counts, and a commutative, idempotent **merge** — because, as Section 4 shows, the entire persistence and multi-writer story rests on canonicity being exact rather than approximate.

Relationship to ontologies. OntoDAG sits deliberately between flat tags and full ontology stacks. It is an ontology in Gruber’s sense — an explicit specification of a conceptualization [8] — but it keeps only the taxonomic backbone: the subsumption order, which in OWL terms is the `subclassOf` skeleton [16]. It omits properties, axioms, restrictions, and description-logic reasoning, and in exchange its one query primitive is cheap, predictable, and needs no reasoner. The reference implementation imports and exports OWL class hierarchies (including Manchester syntax), so existing ontologies can seed an OntoDAG and curated OntoDAGs can flow back to standard tools.

4 Persistence on Swarm

4.1 Swarm in one paragraph

Ethereum Swarm [15] is a peer-to-peer storage network whose atomic unit is an immutable 4KB *chunk*, addressed by the hash of its content. Arbitrary-length payloads are split into Merkle trees of chunks behind a single 32-byte reference. Storage is paid for with *postage stamps* — prepaid, on-chain batches that entitle chunks to live in the network until the batch’s

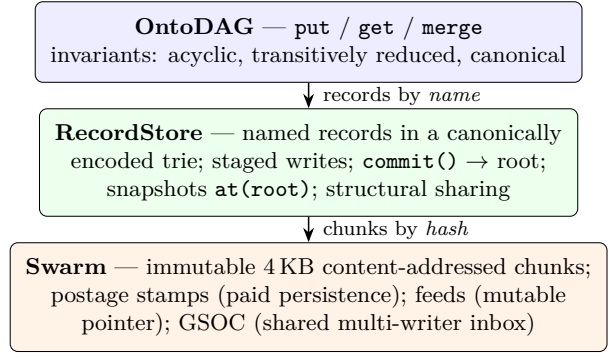


Figure 3: The three-layer architecture. Graph semantics exist only in the top layer; the record store knows nothing about edges or descendants and is reusable for any versioned key-value data; the bottom layer is the public Swarm network. Each layer talks to the next through a narrow interface, so storage-layout changes (chunk packing, batching) never touch graph code.

value is exhausted; unstamped or expired chunks are eventually garbage-collected. *Feeds* provide mutability: a single-owner, signed pointer that can be updated to reference the latest immutable state. *GSOC* (Graffiti Several Owner Chunks) generalizes this into a shared address to which many parties can post signed messages — a decentralized inbox we will use in Section 5.

4.2 Why the fit is unusually good

Content-addressed immutable storage merges two senses of “persistent” that normally have to be engineered separately: *durable* (the database sense) and *immutable with structural sharing* (the functional-programming sense, as in Git or Ethereum’s state trie). For most data structures this is merely convenient. For OntoDAG it is foundational, because of the canonical form: since the graph is a pure function of its content, and the encoding below is deterministic, **the same knowledge always hashes to the same root reference**. Consequences, in decreasing order of obviousness:

- **Versions are free.** Every `commit` yields a new root; old roots remain valid, self-consistent snapshots forever. History is a by-product, not a feature to build.
- **Diffs are cheap.** Two roots share every unchanged subtree by construction, so comparing versions touches only what differs — the Merkle-tree argument [11] applied to an ontology.
- **Equality is a hash comparison.** Two parties can verify they hold the same ontology — however differently they arrived at it — by comparing 32 bytes. This is the property the multi-writer story in Section 5 cashes in.

4.3 The RecordStore layer

Between the graph and the chunk store sits a deliberately generic layer (Figure 3): a versioned key→record store over any content-addressed chunk backend. It contributes exactly what Swarm does not natively provide: typed records rather than raw references; a *canonical codec* (sorted keys, fixed separators — a correctness requirement, since any encoding nondeterminism would break same-content-same-root); *atomic commits* (an OntoDAG put touches several records — the new node, its parents’ edge lists — and `commit()` lands them as one new root, so no reader ever sees a torn write); and *snapshot isolation* (a reader pins one root and traverses a frozen, self-consistent dataset with no locking). Keys live in a compacted radix trie whose nodes are themselves canonically encoded chunks, giving structural sharing: in the test suite, updating 1 of 200 records writes fewer than 12 new chunks.

One OntoDAG node maps to one record, keyed by its name:

```
"Germany": {
  "up":    ["Europe"],           // parents,
    sorted
  "down": ["ad#1", "ad#2"],      // children,
    sorted
  "count": 2,                   // descendants
  "payload": null,              // optional
    Swarm ref
  "meta": {}                    // optional, e.g.
    type
}
```

A typical record is 300–500 bytes; hub nodes with thousands of edges exceed a chunk, but Swarm’s splitter already turns any payload into a chunk tree behind one reference, so the schema stays flat.

4.4 Performance model

The dominant cost is the network chunk fetch, not any code-level choice: a chunk on the local Bee node costs 1–5 ms, a cold chunk fetched across the network 100–500 ms — four to six orders of magnitude above an in-memory pointer hop. Two facts make this workable. First, immutable chunks *cache perfectly*: no invalidation is ever needed, at any layer (in-process, the local Bee store, HTTP caches downstream). Second, ontologies are small relative to the content they organize — a 100,000-node OntoDAG is a few tens of megabytes — so the practical pattern is *hydrate once, query in RAM, commit mutations for durability and sync*. Partial, on-demand loading through the trie is the regime for graphs that outgrow memory, and the layering of Figure 3 confines that change below the record interface.

4.5 Implementation status

The full stack of Figure 3 is implemented in Python and open source [6]: the invariant-checked data structure, the record store (unit-tested and model-based-fuzzed

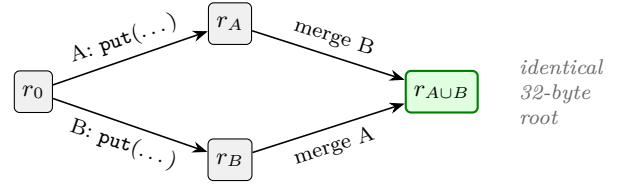


Figure 4: Convergent merge. Writers A and B extend the same snapshot r_0 independently; whichever order the merges happen in — and however many times they are repeated — both replicas reach the *same* root reference, verifiable by comparing hashes. No coordinator, no lock, no conflict dialog.

against a dictionary oracle, including the canonical-root property under arbitrary put/delete histories), and the `SwarmOntoDAG` adapter (13 tests, including merge convergence to bit-identical roots). The chunk backend has passed its integration suite against a live Bee v2.8.1 light node on Gnosis mainnet with a real purchased postage batch.

5 Concurrent writers without coordination

Git taught a generation of developers that immutable snapshots plus merge beat locks — but Git merges can conflict, because arbitrary text has no merge rule. OntoDAG’s canonical form supplies one. Define the canonical state of a replicated OntoDAG as

$$G = \text{TR} \left(\bigcup_{w \in \text{writers}} E_w \right),$$

the transitive reduction of the union of all asserted edges. Union is commutative, associative, and idempotent, and TR is a unique normal form [1] — so concurrent put operations from independent writers can be applied in any order, or repeatedly, and every replica converges to the same graph and hence (via the canonical encoding) to the same root hash (Figure 4). These are precisely the convergence conditions of state-based CRDTs [14]. Removal, as always in CRDT territory, needs care: a plain delete does not commute with a concurrent re-add, so removals carry observed-remove tombstones.

The planned transport is Swarm-native: writers post signed operations (`put(name, supers)`, `remove(name)`) to a shared GSOC address, and any reader deterministically folds the pending operation set into the current root and publishes the result on a feed. Because folding is deterministic, it does not matter *who* folds — any two folders produce bit-identical roots.

The payoff for users is the local-first property [10]: **personal categories extend shared ontologies without corrupting them**. A community can maintain a shared base taxonomy (say, the car market of Figure 2); an individual layers private categories over

it (*my shortlist, seen at the dealer*); merge semantics guarantee the overlay never damages the base, and the base can be upgraded under the overlay by the same union-and-reduce rule.

6 Where do categories come from? Learning by compression

Everything so far assumes someone *asserts* the categories. Manual curation is exactly right for personal, deliberate structure, but it does not scale to large corpora, and it never discovers the categories you did not think to create. The mathematical home of “categories latent in data” is Formal Concept Analysis [17, 7]: given a binary object×attribute table, FCA derives all *concepts* — maximal sets of objects sharing a maximal set of attributes — ordered by subsumption into a lattice: multi-parent, acyclic, downward-closed — the same shape as an OntoDAG. FCA’s classical defect is that it is *too* faithful: the number of formal concepts can grow exponentially, and most of them are coincidences of the sample rather than structure worth keeping.

The minimum description length (MDL) principle [13, 9] supplies the missing filter, and it can be stated entirely in terms a systems audience already trusts: **the best model of your data is the one that compresses it best, where the model’s own description counts against it.** Formally, choose the concept DAG G minimizing

$$L(G) + L(D | G),$$

the bits to describe the model plus the bits to describe the data given the model. A candidate concept (say, *German electric car*) earns a place in the DAG only if factoring it out shortens the total description: each object covered by it can then be described as “one of those, plus these deviations,” and the saving must exceed the cost of writing the concept down (Figure 5). Rare coincidences never pay for themselves; genuine regularities always do, given enough data. This is not an analogy — compression and prediction are formally two views of the same quantity — but the practical reading is the honest one: *MDL is a principled, parameter-free answer to “is this category worth keeping?”*

The companion project **mdl-fca** [5] implements this program: a greedy pair-merge constructor proposes candidate concepts and a search framework keeps those that pay their way, in both batch and online settings. Crucially, its output DAG is structurally aligned with OntoDAG — downward-closure semantics, multiple parents, acyclic — so a learned DAG becomes an OntoDAG snapshot through an essentially identity conversion, and everything in Sections 4–5 (canonical roots, snapshots, merge) applies to learned structure unchanged. The learner remains a separate package

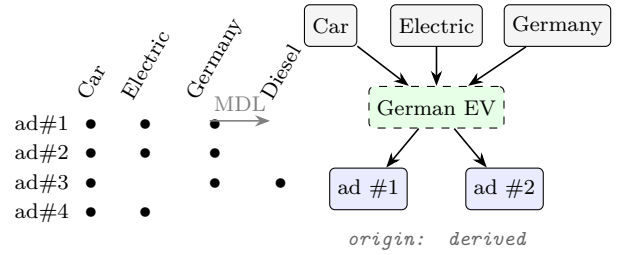


Figure 5: Learning a category by compression. The attribute combination {Car, Electric, Germany} recurs in the object×attribute data (left), so factoring it out as a concept shortens the total description: the learner inserts a *German EV* node (dashed) into the DAG, and the covered items are described through it. A combination that occurred once would not pay for its own description and is never materialized.

that depends on OntoDAG as a library, so learners are swappable and the core stays dependency-free.

7 Learned categories are a cache: MDL meets storage economics

Mixing hand-asserted and machine-proposed structure in one graph raises an obvious worry: which nodes can you trust, and which can you afford to lose? The answer is not an instance/class distinction (rejected in Section 3) but *provenance*, orthogonal to what a node is:

- **origin: asserted** — placed by an external act (a user, an import, an observation). Ground truth; not regenerable; losing it is data loss.
- **origin: derived** — proposed by a learner because it compressed asserted data. A pure function of (corpus, learner); pinned as `derived_from = (corpus_root, learner_version)`, where the corpus root is itself a content-addressed snapshot. Losing it is only a cache miss.
- **endorsed** — a user may promote a derived concept they have come to rely on; re-learning may propose a replacement but must not silently overwrite an endorsed node.

This single bit is where the learning story and the storage economics click together. Swarm charges rent per chunk via postage stamps (Section 4); every node record is a chunk; so every category has a carrying cost. Provenance says which costs are obligatory: **asserted nodes must stay stamped and pinned regardless; derived nodes are kept warm only while query traffic justifies their postage**, and are otherwise allowed to expire and be recomputed on demand — *exactly* the economics of a cache, applied to knowledge. The same bit routes synchronization in the shared/personal architecture of Section 5 (Table 1): the big shared base is largely derived and therefore

	shared	personal
asserted	curate, stamp durably	back up; irreplaceable
derived	recompute from corpus; sync diffs only	recompute per device; let postage lapse

Table 1: Provenance $\times$ ownership routes what must be stored durably versus what may be lazily recomputed.

cheap to distribute (peers with the corpus recompute it and sync only diffs), while personal assertions are the small, irreplaceable payload that deserves durable stamps and backup.

There is also a genuinely new research question here, which we record without claiming to have solved it. Classical MDL scores a concept by how often it helps *generate* the data. In a metered, separately-queried store, a concept also has a *retrieval* value — how often queries actually read it — which is a property of the workload, not the corpus. The natural extension is

$$L(G) + L(D | G) + \lambda \sum_{c \in G} f_{\text{ret}}(c) \cdot s(c),$$

where f_{ret} comes from the query log, $s(c)$ is the storage cost of keeping c warm, and λ (bits per byte-second) prices storage against description length. The term vanishes in the in-memory setting and collapses when the query distribution mirrors the data — but personal tools are exactly where the two diverge, as users repeatedly query their statistically-rare pet categories. Retrieval-aware MDL would let the learner itself decide which categories deserve postage.

8 Status and future work

Implemented and tested [6]: the invariant-checked core (acyclicity, unique transitive reduction, order independence, iterative traversals, commutative idempotent merge); the record store with canonical roots verified by model-based fuzzing; the Swarm adapter with merge-convergence-to-identical-roots tests; OWL import/export; a REST API and web demo; Bee integration validated on Gnosis mainnet (v2.8.1, real postage batch). Open, in planned order: network-level retrievability and pinning checks; the GSOC operation log and fold rule of Section 5 (the in-memory merge algebra it needs is already tested); a signed feed pointer; chunk-packing and batched fetches (behind the existing interface, driven by usage data rather than speculation); and the mdl-fca integration with the provenance schema of Section 7.

9 Conclusion

The organizational layer of storage has lagged its transport layer by decades: we address content by cryptographic hash and then file it in a tree from 1970. OntoDAG proposes a small correction with large consequences. A subsumption-only, multi-parent category DAG is expressive enough to combine hierarchical

and faceted classification, yet simple enough to have a *canonical form* — and on a content-addressed network, canonical form is superpower: identical knowledge gets identical hashes, versions and diffs come for free, and uncoordinated writers provably converge. The same structure is the natural output format of compression-based concept learning, and Swarm’s storage economics gives learned categories precisely the lifecycle they deserve: asserted knowledge is preserved like data, derived knowledge is priced like a cache. Between the directory tree and the description-logic reasoner there is a productive middle — and it fits in 4 KB chunks.

Acknowledgments. The reference implementation, its Swarm design document, and the mdl-fca learner are open source at <https://github.com/petfold/ontodag> and <https://github.com/petfold/mdl-fca>.

References

- [1] A. V. Aho, M. R. Garey, and J. D. Ullman. The transitive reduction of a directed graph. *SIAM Journal on Computing*, 1(2):131–137, 1972.
- [2] J. Benet. IPFS — content addressed, versioned, P2P file system, 2014. arXiv:1407.3561.
- [3] V. Bush. As we may think. *The Atlantic Monthly*, 176(1):101–108, 1945.
- [4] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [5] P. Földiák. mdl-fca: Minimum-description-length concept learning over binary data, 2026. <https://github.com/petfold/mdl-fca>.
- [6] P. Földiák. OntoDAG: A DAG-based associative memory and category manager, 2026. <https://github.com/petfold/ontodag>.
- [7] B. Ganter and R. Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer, Berlin, 1999.
- [8] T. R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.
- [9] P. D. Grünwald. *The Minimum Description Length Principle*. MIT Press, Cambridge, MA, 2007.
- [10] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, pages 154–178, 2019.
- [11] R. C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO ’87*, volume 293 of *LNCS*, pages 369–378. Springer, 1988.
- [12] S. R. Ranganathan. *Colon Classification*. Madras Library Association, Madras, 1933.
- [13] J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

- [14] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *LNCS*, pages 386–400. Springer, 2011.
- [15] V. Trón. *The Book of Swarm: Storage and Communication Infrastructure for a Self-Sovereign Digital Society*. Swarm Foundation, 2020. <https://www.ethswarm.org/the-book-of-swarm.pdf>.
- [16] W3C OWL Working Group. OWL 2 Web Ontology Language Document Overview (Second Edition), 2012. W3C Recommendation, <https://www.w3.org/TR/owl2-overview/>.
- [17] R. Wille. Restructuring lattice theory: An approach based on hierarchies of concepts. In I. Rival, editor, *Ordered Sets*, pages 445–470. Reidel, Dordrecht, 1982.